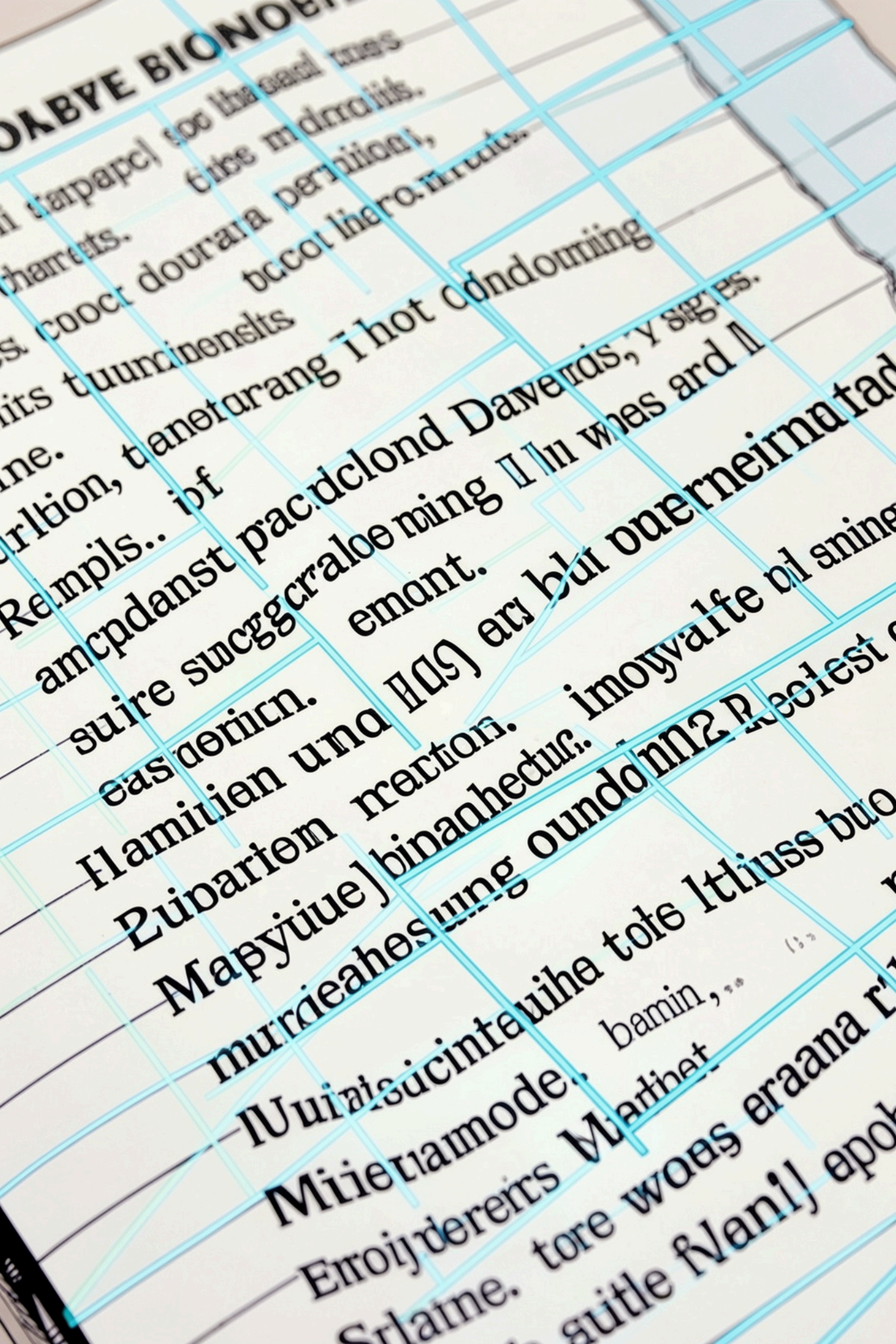




Optical Character Recognition (OCR)

A technical project presentation on automated text extraction from images and documents

Guided by: Leang Vakhim · **Presented by:** Ry Vorovath

Table of Contents

01	System Overview
02	Technical Stack
03	Model Architecture
04	Data Processing Pipeline
05	Demo & Testing
06	Challenges & Future Improvements

System Overview

This project builds an end-to-end OCR recognizer that automatically detects and extracts text from images, scanned documents, and natural scenes. The system transforms unstructured visual data into machine-readable, searchable text — enabling downstream applications such as document digitization, data entry automation, and accessibility tools.

Input

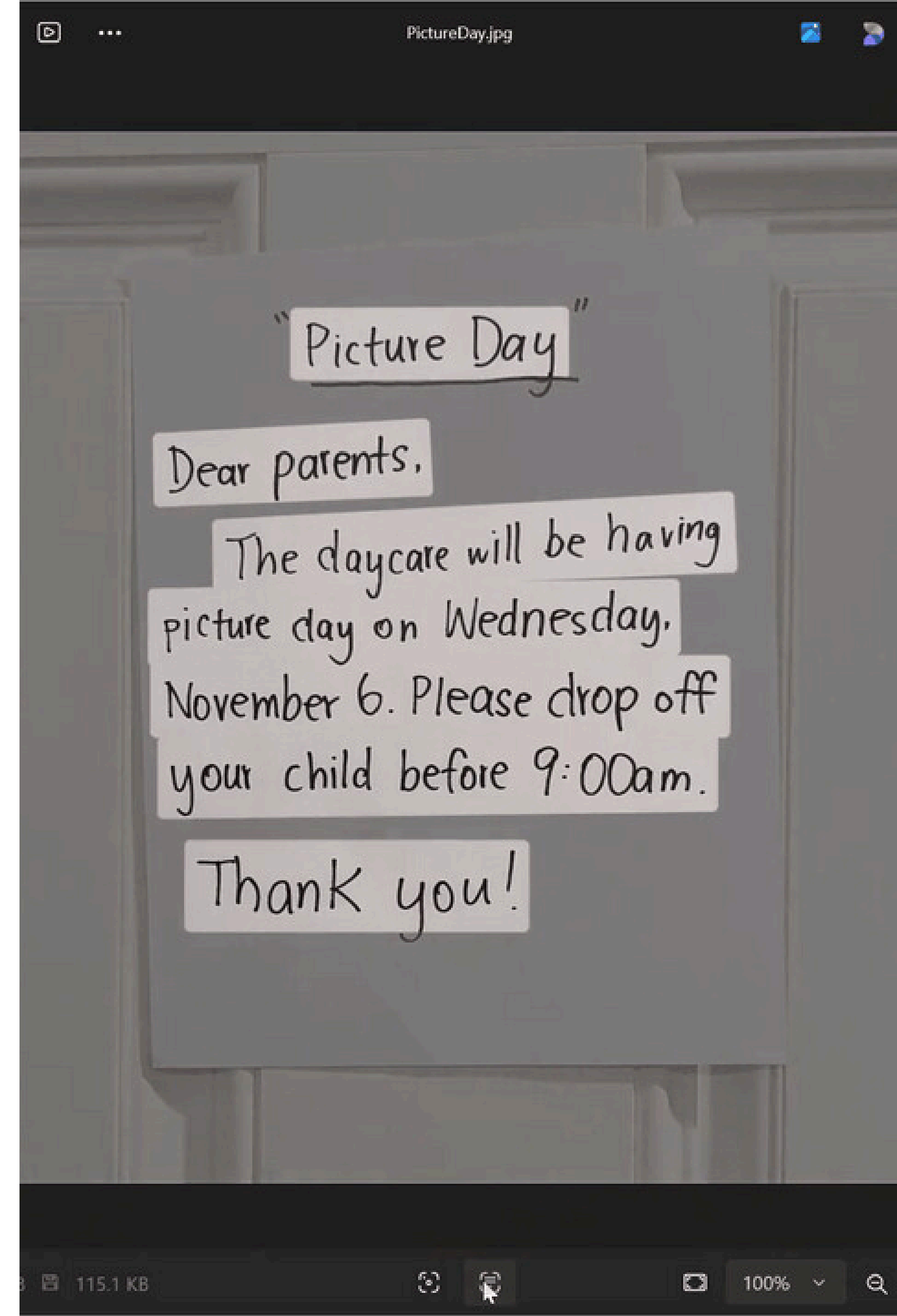
Images, PDFs, and scanned documents

Process

Preprocessing → Detection → Recognition

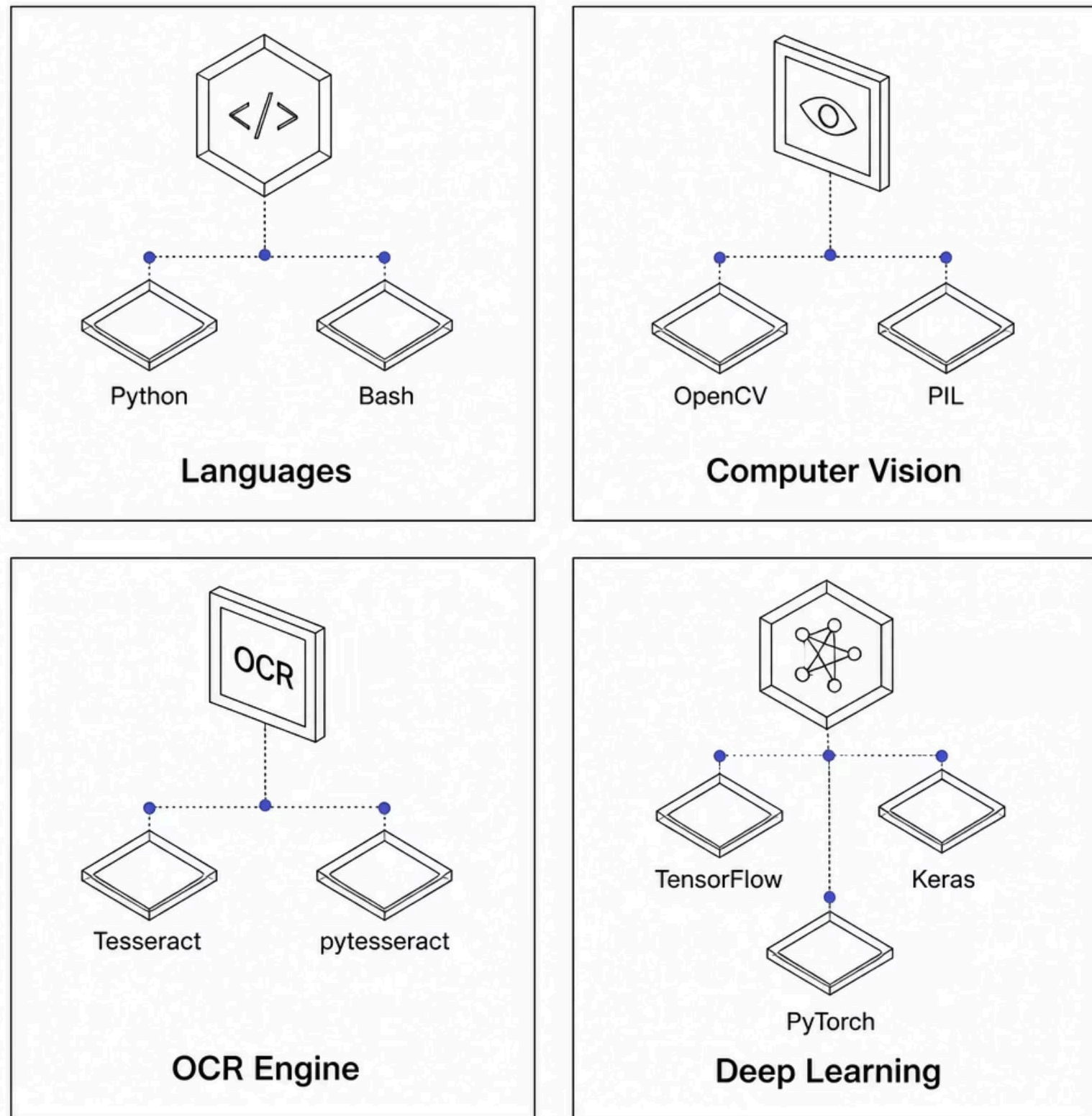
Output

Structured, searchable text data



Technical Stack

The project leverages a modern, open-source toolchain optimized for image processing and deep learning.

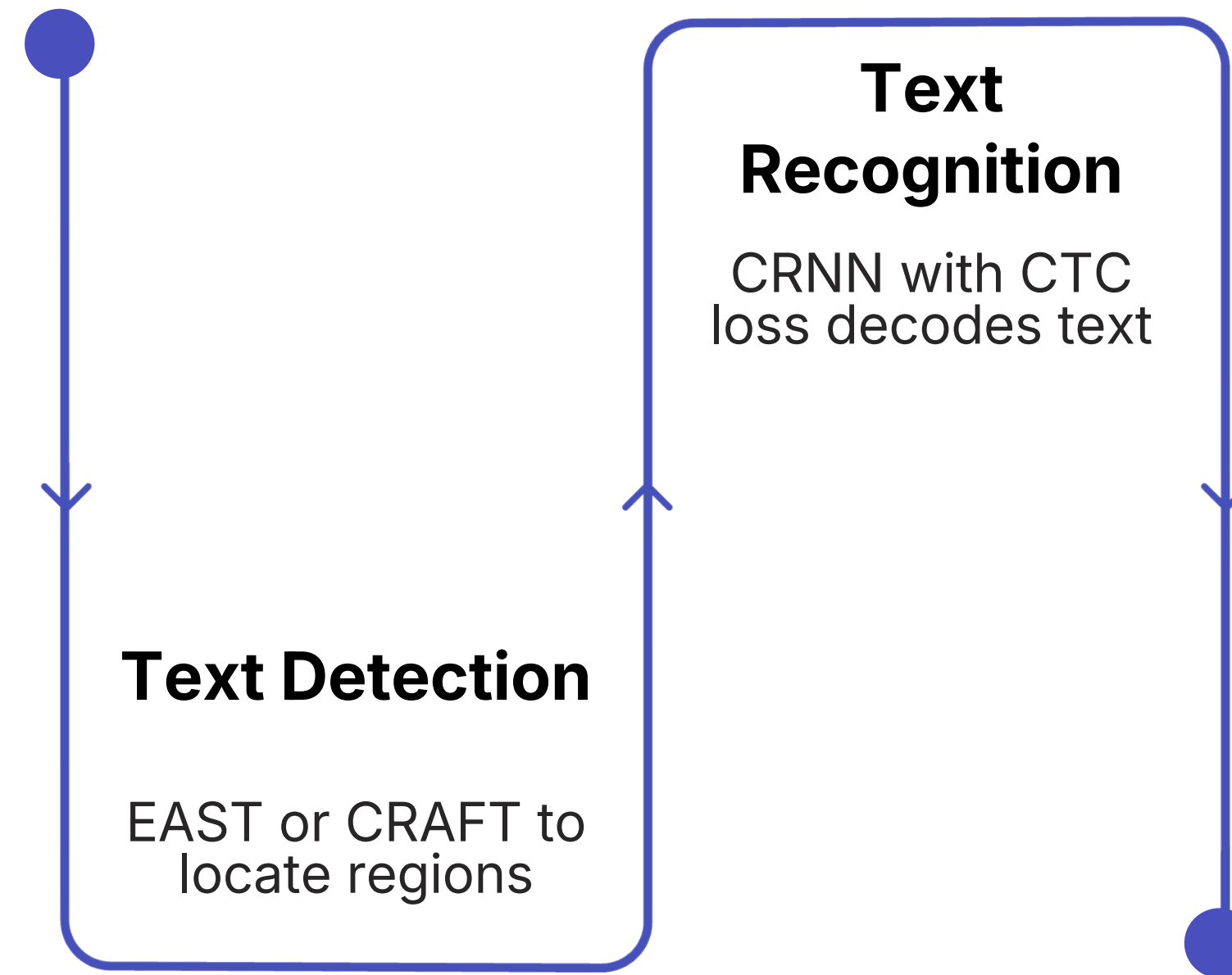


Core Libraries

- **Python** — primary development language
- **OpenCV** — image preprocessing, deskewing, and decoding
- **EasyOCR** — OCR engine (combines detection + recognition, not Tesseract)
- **PyTorch** — deep learning backend that runs the EasyOCR models
- **FastAPI** — backend API framework serving the /process-ocr/ endpoint
- **NumPy** — image array handling between OpenCV and EasyOCR

Model Architecture

The OCR system follows a two-stage approach: **text detection** to locate text regions, followed by **text recognition** to decode characters within those regions.



The recognition module uses a Convolutional Recurrent Neural Network (CRNN) with Connectionist Temporal Classification (CTC) loss, enabling accurate recognition without explicit character segmentation.

TEXT DETECTION USING CRAFT (Character Region Awareness for Text Detection)

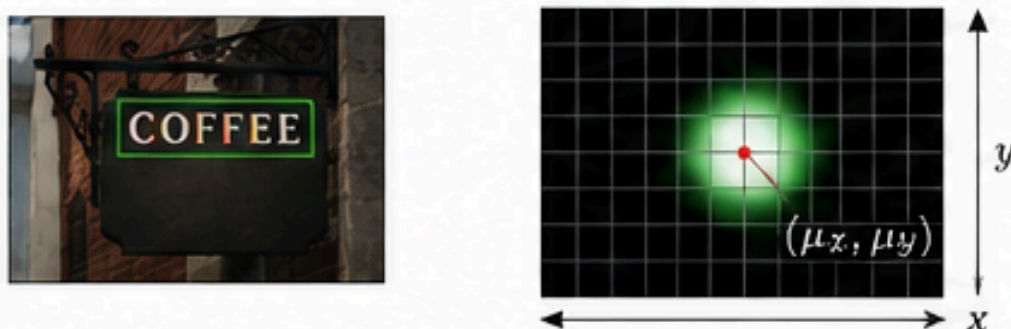
CRAFT does not just locate entire words; it focuses on individual characters. It predicts two key scores:

1 Region Score: Determines whether a specific area contains a character.

2 Affinity Score: Determines the relationship between adjacent characters, allowing the system to group characters into meaningful words.

A. Gaussian Heatmaps for Region Score

For each character, CRAFT generates a Gaussian heatmap centered at the character's center (μ_x, μ_y) .



The value at any point (x, y) is defined by a Gaussian distribution:

$$f(x, y) = \frac{1}{2\pi\sigma^2} e^{-\frac{(x - \mu_x)^2 + (y - \mu_y)^2}{2\sigma^2}}$$

where:

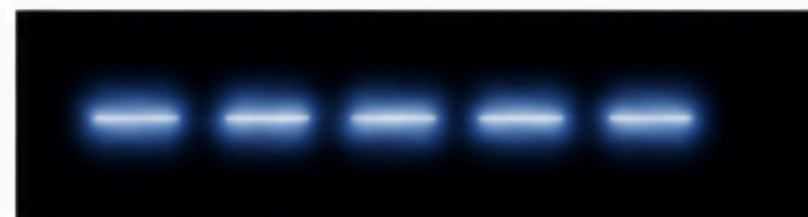
- (μ_x, μ_y) = center of the character
- σ = standard deviation (controls spread of the Gaussian)
- $f(x, y)$ = value (probability) at position (x, y)

B. Affinity Score (Character Relationship)

Affinity score predicts the relationship (link) between adjacent characters.



Affinity Heatmap (example)

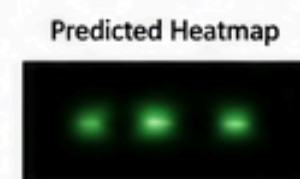


High affinity values (bright areas) indicate that two characters belong to the same word.

C. Loss Function (Training Objective)

CRAFT uses Mean Squared Error (MSE) to measure the difference between the predicted heatmap and the ground-truth heatmap.

Region Score Loss



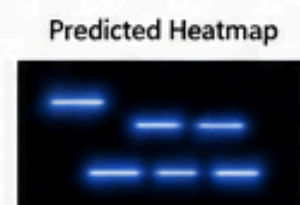
vs



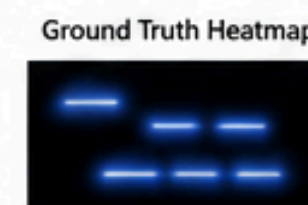
MSE Loss

$$L_{region} = \frac{1}{N} \sum_{i=1}^N (P_i - G_i)^2$$

Affinity Score Loss



vs



MSE Loss

$$L_{affinity} = \frac{1}{N} \sum_{i=1}^N (P_i - G_i)^2$$

where:

- P_i = predicted value at pixel i
- G_i = ground-truth value at pixel i
- N = total number of pixels

Total Loss

$$L = \lambda_{region} L_{region} + \lambda_{affinity} L_{affinity}$$

Summary: CRAFT detects text by locating individual characters (Region Score) and linking them together (Affinity Score) to form meaningful words and text bounding boxes.

TEXT RECOGNITION USING CRNN (Convolutional Recurrent Neural Network) – EasyOCR

Concept: This stage transforms the cropped image regions (detected by CRAFT) into a sequence of characters. It consists of three primary components:

1 Convolutional Layers (CNN):

Extracts specialized feature maps from the image snippets.

2 Recurrent Layers (RNN/LSTM):

Interprets the sequence of features from left to right, capturing the context of characters within a word.

3 Transcription Layer (CTC):

Translates the sequential output from the RNN into final text.

Model Used in EasyOCR:

english_g2.pth

Input from Text Detection (CRAFT)

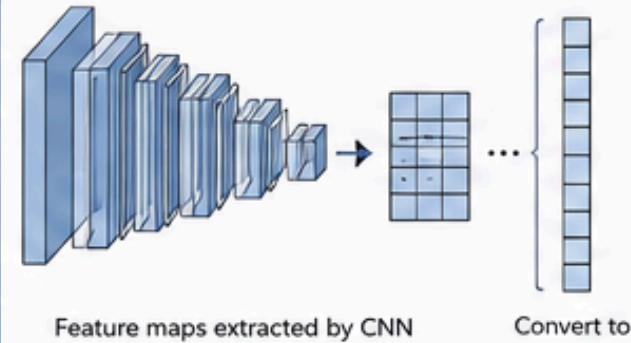


Detected text region

Cropped Region

COFFEE

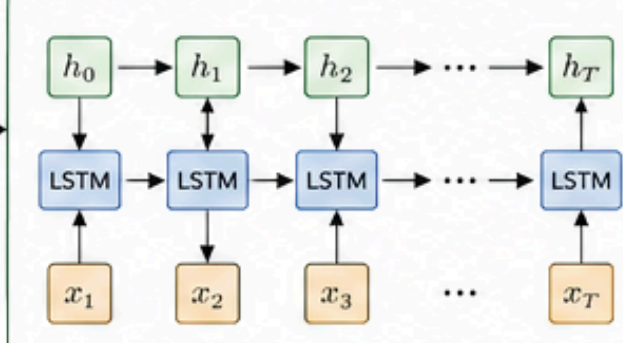
1. CNN (Feature Extraction)



Feature maps extracted by CNN

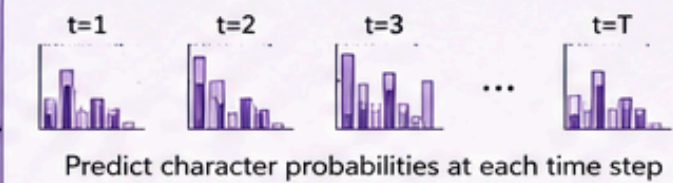
Convert to 1D sequence

2. RNN / LSTM (Sequence Modeling)



Read feature sequence from left to right

3. CTC (Transcription Layer)



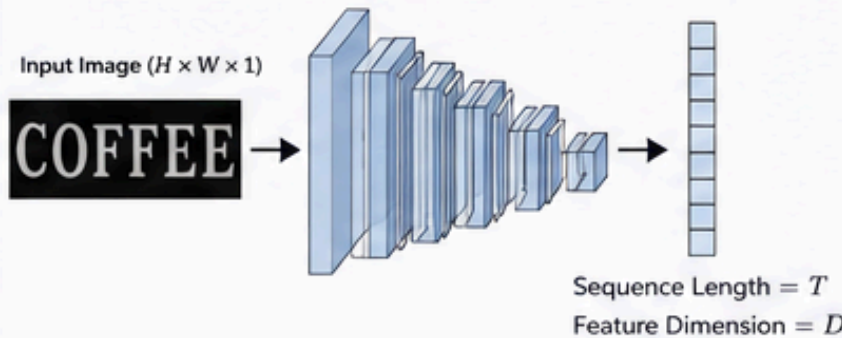
Predict character probabilities at each time step

CTC Decoding

C O F F E E

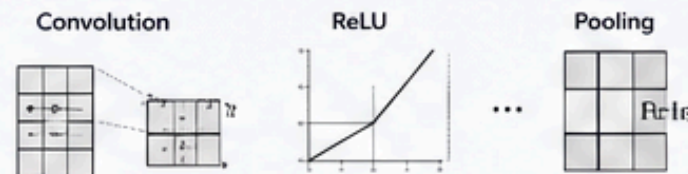
A. Convolutional Layers (CNN)

CNN extracts hierarchical visual features from the input image. The output is a sequence of feature vectors (from left to right).



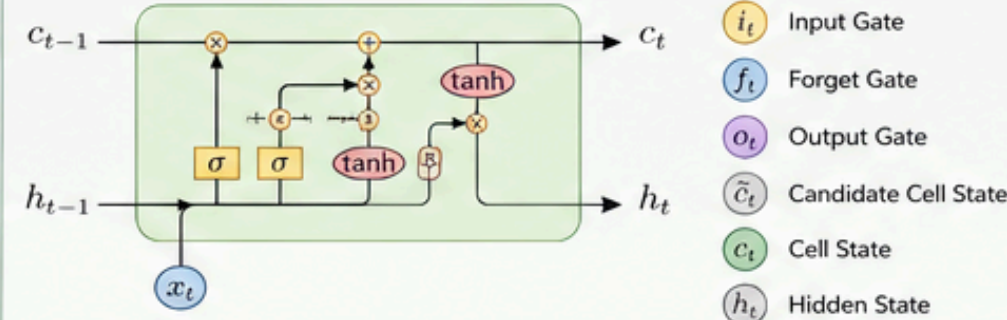
Sequence Length = T
Feature Dimension = D

CNN Operations (example)



B. Recurrent Layers (RNN / LSTM)

LSTM reads the feature sequence and captures context of characters using its internal gates.



LSTM Gate Equation (example: Forget Gate)

The forget gate decides what information to discard from the cell state.

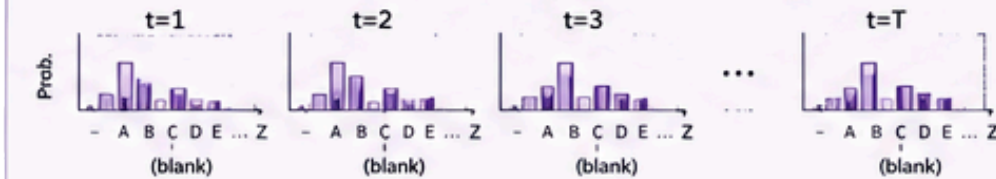
$$f_t = \sigma(W_f \cdot [h_{t-1}, x_t] + b_f)$$

where:

- f_t : forget gate at time t
- σ : sigmoid activation function
- W_f : weight matrix
- h_{t-1} : previous hidden state
- x_t : current input
- b_f : bias

C. Transcription Layer (CTC)

CTC (Connectionist Temporal Classification) computes the probability of all possible alignments between the predicted sequence and the target text without requiring character-level alignment.



CTC Loss

CTC calculates the total probability of all valid paths π that result in the correct target text y .

$$P(y|x) = \sum_{\pi \in B^{-1}(y)} P(\pi|x)$$

$$\text{CTC Loss} = -\log(P(y|x))$$

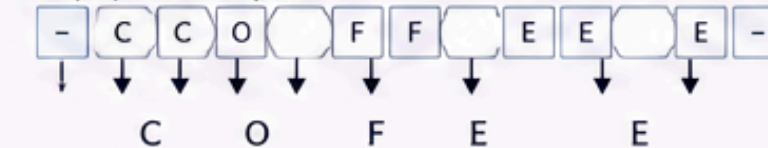
where:

- x : input feature sequence
- y : target text
- π : a possible alignment path
- $B^{-1}(y)$: set of all paths that collapse to y

CTC Decoding (Greedy Search example)

Remove repeated characters and blanks.

Example prediction sequence:



Final Output:

C O F F E E

How It Works (End-to-End)

- CRAFT detects text regions and crops them.
- CNN extracts visual features from the cropped image.
- LSTM reads the features from left to right and understands the sequence of characters.
- CTC converts the sequential predictions into final text.

Key Points

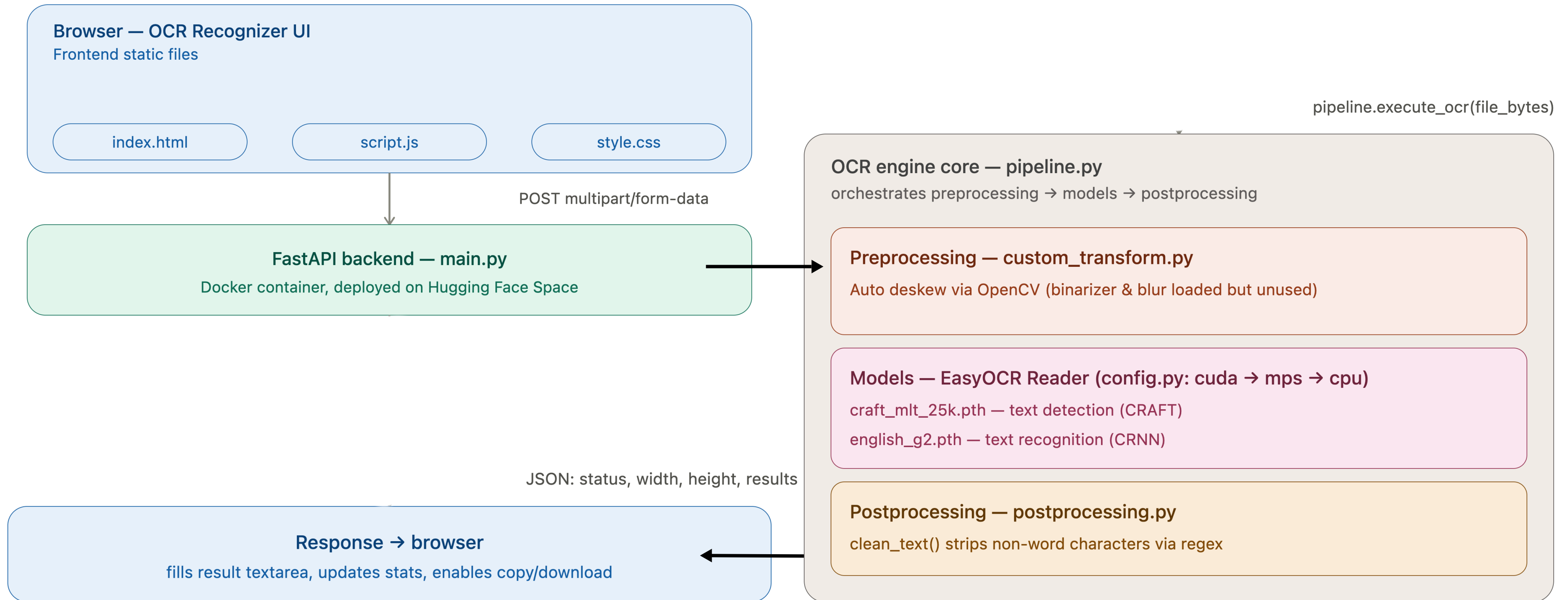
- ✓ CNN captures "what" (visual features of characters).
- ✓ LSTM captures "in what order" (context & sequence).
- ✓ CTC allows training without character-level alignment, making the system robust for OCR.

Model Used

english_g2.pth

Pre-trained CRNN model in EasyOCR for English text recognition.

Data Processing Pipeline



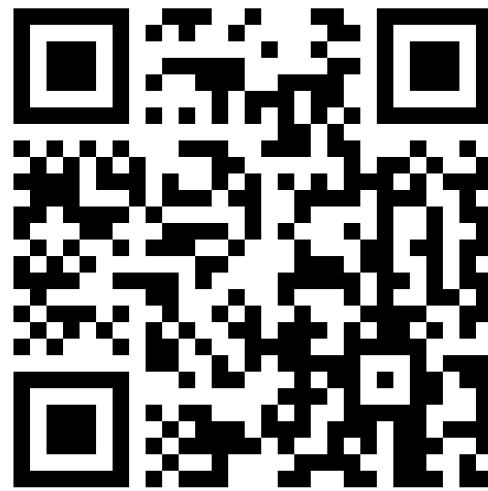
Demo

The live demonstration showcases the OCR system processing real-world inputs — from clean scanned documents to complex natural scene images with varied fonts, lighting, and orientations.

Link of the web

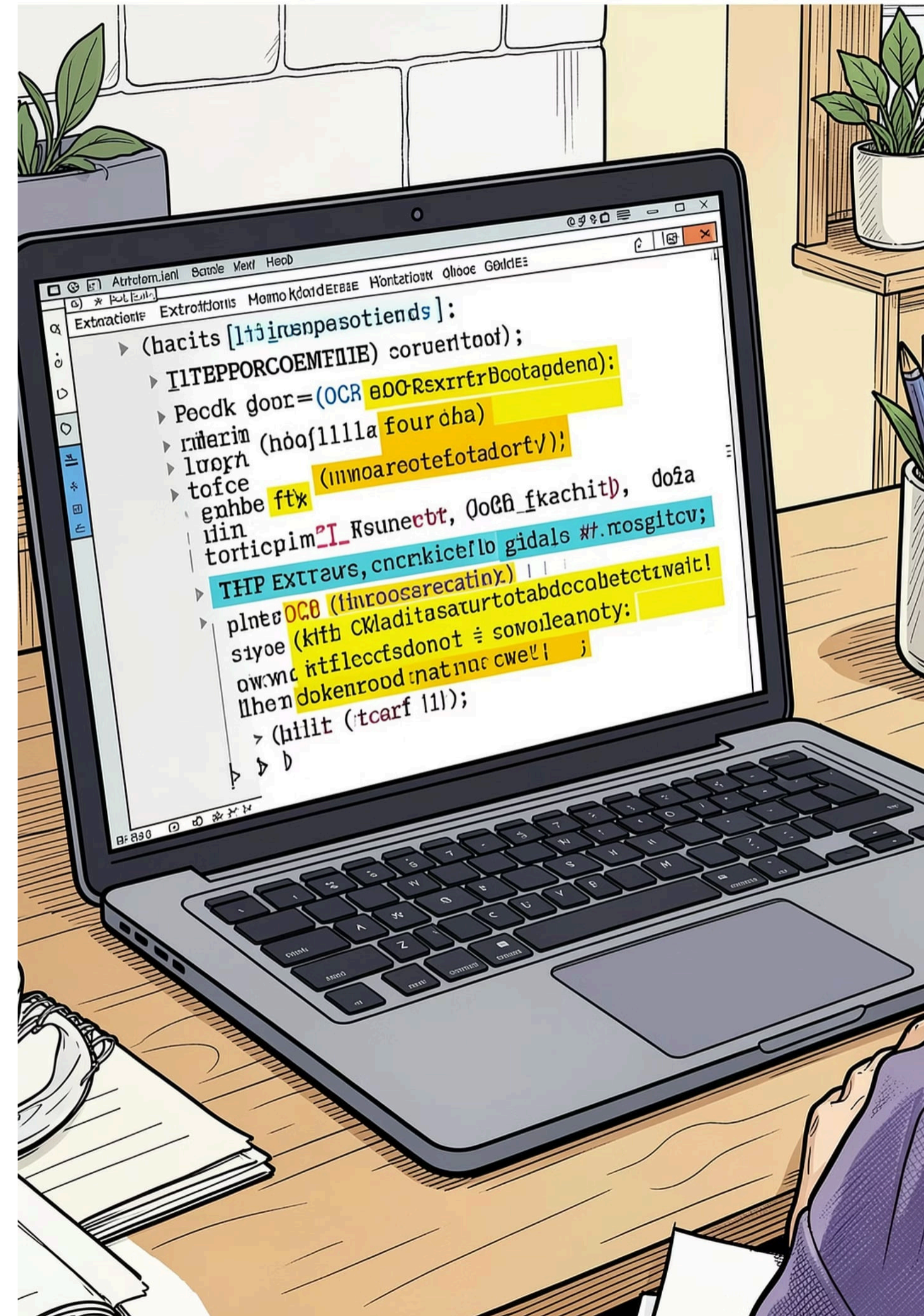
https://vath7677.github.io/web_ocr/

QR-Code for Testing



Inference Speed

Real-time processing on standard hardware with batch support for throughput



Challenges & Future Improvements

Current Challenges

- **Low-quality images** — blur, glare, and low resolution reduce accuracy
- **Complex scripts** — non-Latin alphabets and mixed-language text require specialized training
- **Handwritten text** — high variability in stroke and style limits recognition
- **Curved or distorted text** — standard detectors struggle with non-horizontal layouts

Roadmap

- Integrate transformer-based models (e.g., TrOCR) for improved context awareness
- Expand language support to Khmer, Chinese, and France scripts
- Deploy as a REST API or web service for broader accessibility
- Add layout analysis to preserve document structure and formatting

Thank you !!